

Analysis and Design of Algorithms

算法分析与设计

Chapter 1: Introduction



School of Software Engineering © Ye Luo



软件工程学院 ©

叶洛

Introduction of Teacher

Contact Information

Office: Jishi Building, Room 314

Email: yeluo@tongji.edu.cn

Course Information:

Slides can be downloaded from:

 QQ 群 共享文件

TA Information:

教师介绍

联系方式

办公室: 济事楼314室

邮箱: yeluo@tongji.edu.cn

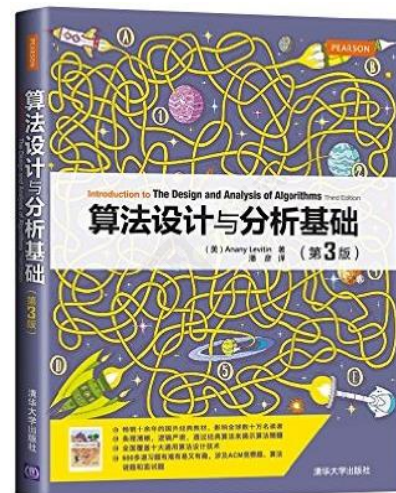
 课程信息: 讲义可从以下链接下载:

 QQ群共享文件

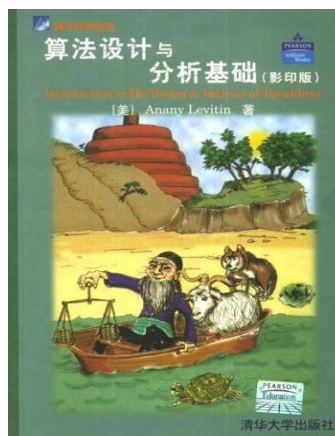
TA信息:

Our References

算法设计与分析基础(第3版).
(美) Anany Levitin 著, 潘彦译.
清华大学出版社 .2015年2月.

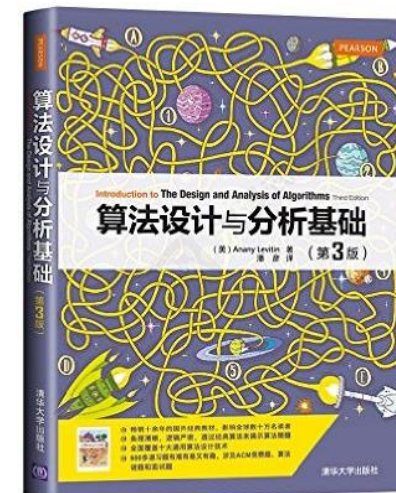


Introduction to the Design and Analysis
of Algorithms.
Anany Levitin.
清华大学出版社. 2003年.

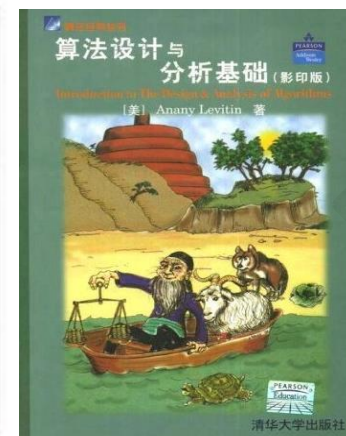


我们的参考文献

算法设计与分析基础(第3版)
(美)Anany Levitin 著,潘彦译.
清华大学出版社 .2015年2月.



算法设计与分析导论。
Anany Levitin。
清华大学出版社. 2003年.



Survey of Reference Books Worldwide

编号	教材	使用次数
[1]	Introduction to Algorithms, by Cormen, Leiserson, Rivest, and Stein.	32
[2]	Algorithm Design, by Kleinberg and Tardos.	20
[3]	Algorithms, by Dasgupta, Papadimitriou and Vazirani.	8
[4]	Algorithms, by Sedgewick and Wayne.	1
[5]	An Introduction to Algorithms: their Methods and Madness, by A.R.Siegel.	2
[6]	Data Structures and Network Algorithms, by R. E. Tarjan.	1
[7]	Randomized Algorithms, by Motwani and Raghavan.	2
[8]	Programming Pearls, by Bentley.	1
[9]	Introduction to Algorithms: a Creative Approach, by Manber.	1
[10]	Probability and Computing, by Mitzenmacher and Upfal.	1
[11]	The Probabilistic Method, by Alon and Spencer.	2
[12]	The Design of Approximation Algorithms, by Williamson and Shmoys.	2
[13]	The Art of Computer Programming, by Knuth.	2
[14]	Algorithmics, by Harel.	2
[15]	Data Structures and Algorithm Analysis, by Shaffer.	1
[16]	Algorithms in C, by Sedgewick.	1
[17]	Computer Algorithms, by Baase and Gelder.	1
[18]	The Design and Analysis of Computer Algorithms, by Aho, Hopcroft and Ullman.	1
[19]	Introduction to The Design and Analysis of Algorithms, by Levitin.	1
[20]	Algorithms, by Johnsonbaugh.	1
[21]	Approximation Algorithms, by Vazirani.	1
[22]	Algorithms Unlocked, by Cormen.	1
[23]	Data Structures and Algorithm Analysis in Java, by Weiss.	1
[24]	Fundamentals of Algorithms, by Brassard and Bratley.	1
[25]	Introduction to the Theory of Computation, by Sipser.	1

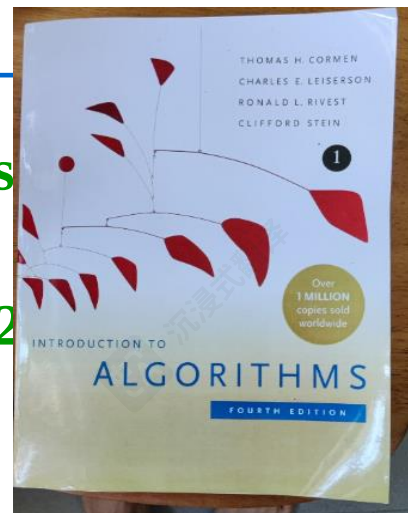
全球参考书目调查

编号	教材	使用次数
[1]	Introduction to Algorithms, by Cormen, Leiserson, Rivest, and Stein.	32
[2]	Algorithm Design, by Kleinberg and Tardos.	20
[3]	Algorithms, by Dasgupta, Papadimitriou and Vazirani.	8
[4]	Algorithms, by Sedgewick and Wayne.	1
[5]	An Introduction to Algorithms: their Methods and Madness, by A.R.Siegel.	2
[6]	Data Structures and Network Algorithms, by R. E. Tarjan.	1
[7]	Randomized Algorithms, by Motwani and Raghavan.	2
[8]	Programming Pearls, by Bentley.	1
[9]	Introduction to Algorithms: a Creative Approach, by Manber.	1
[10]	Probability and Computing, by Mitzenmacher and Upfal.	1
[11]	The Probabilistic Method, by Alon and Spencer.	2
[12]	The Design of Approximation Algorithms, by Williamson and Shmoys.	2
[13]	The Art of Computer Programming, by Knuth.	2
[14]	Algorithmics, by Harel.	2
[15]	Data Structures and Algorithm Analysis, by Shaffer.	1
[16]	Algorithms in C, by Sedgewick.	1
[17]	Computer Algorithms, by Baase and Gelder.	1
[18]	The Design and Analysis of Computer Algorithms, by Aho, Hopcroft and Ullman.	1
[19]	Introduction to The Design and Analysis of Algorithms, by Levitin.	1
[20]	Algorithms, by Johnsonbaugh.	1
[21]	Approximation Algorithms, by Vazirani.	1
[22]	Algorithms Unlocked, by Cormen.	1
[23]	Data Structures and Algorithm Analysis in Java, by Weiss.	1
[24]	Fundamentals of Algorithms, by Brassard and Bratley.	1
[25]	Introduction to the Theory of Computation, by Sipser.	1

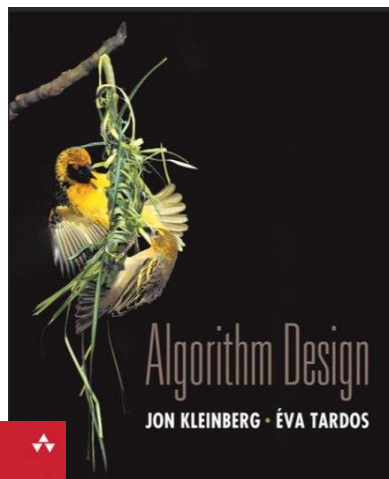
Classical References

Introduction To Algorithms (MIT Press 4th Edition). Thomas H.Cormen.

高等教育出版社& The MIT Press. 2022年. 《算法导论》



Algorithm Design (Pearson Press 1st Edition). Jon Kleinberg, Eva Tardos, Cornell University



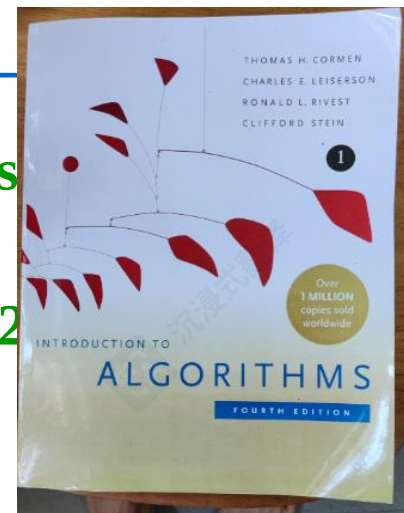
Algorithms (Pearson Press, 4th Edition). Robert Sedgwick, Kevin Wayne, 2011, Princeton University



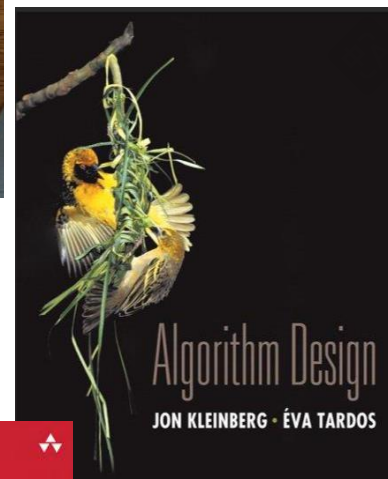
经典参考 References

Introduction To Algorithms (MIT Press 4th Edition). Thomas H.Cormen.

高等教育出版社& The MIT Press. 2022年. 《算法导论》



算法设计 (Pearson Press 1st 版)。乔纳森·克莱因伯格, 伊娃·塔多斯, 康奈尔大学



算法 (Pearson Press, 4th 版)。罗伯特·塞德威克, 凯文·韦恩, 2011年, 普林斯顿大学



Classical Reference 1

Textbook – CLRS Book

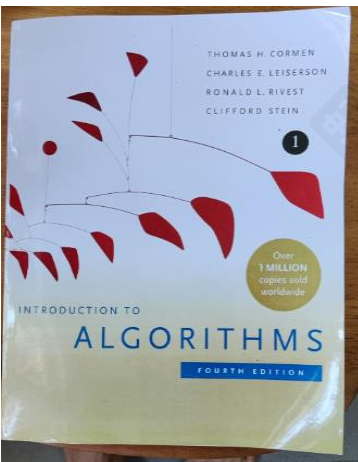
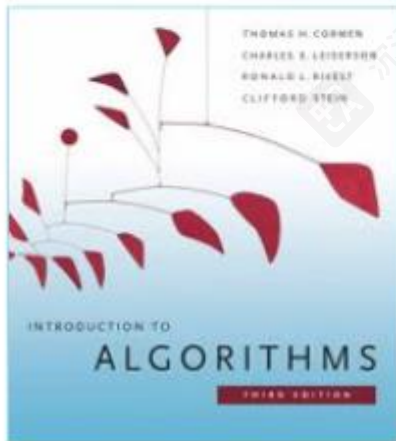
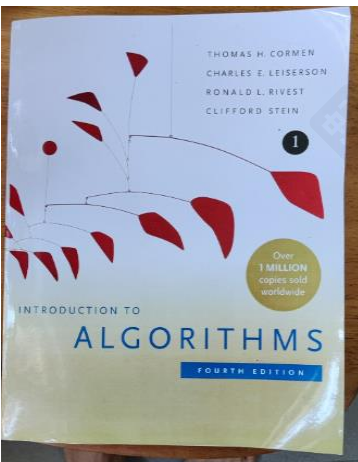
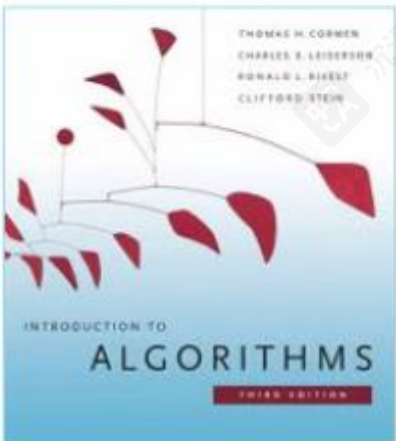
Title: Introduction to Algorithms

Author: T. Cormen, C. Leiserson, R. Rivest, C. Stein

Publisher: The MIT Press, 2022 (First Edition in 1990)

ISBN-10/13: 0262033844 / 978-0262033848

Edition: Second/Third Edition/Forth/Fifth Edition



经典参考1Reference 1

Textbook – CLRS Book

Title: Introduction to Algorithms

Author: T. Cormen, C. Leiserson, R. Rivest, C. Stein

Publisher: The MIT Press, 2022 (First Edition in 1990)

ISBN-10/13: 0262033844 / 978-0262033848

Edition: Second/Third Edition/Forth/Fifth Edition

Classical Reference 1

Universities using CLRS Book

- 24 / Top 40 Univ. use CLRS book (33 courses)



经典参考1Reference 1

Universities using CLRS Book

- 24 / Top 40 Univ. use CLRS book (33 courses)



Classical Reference 2

Textbook – Cornell Book

Title: Algorithm Design

Author: J. Kleinberg, E. Tardos

Publisher: Addison Wesley, 2006.

ISBN-10/13: 0321295358 / 978-0321295354



经典参考2Reference 2

Textbook – Cornell Book

Title: Algorithm Design

Author: J. Kleinberg, E. Tardos

Publisher: Addison Wesley, 2006.

ISBN-10/13: 0321295358 / 978-0321295354



Classical Reference 2

The Authors – Cornell Book



Jon Kleinberg
Cornell
(Ph.D. from MIT)



Éva Tardos
Cornell University
(Hungarian mathematician, **Gödel Prize**)

- 1st Edition: 2006 – Coauthored with Éva Tardos
- 2nd Edition: 2013 – Sole Author

Jon Kleinberg's younger brother → Robert Kleinberg@Cornell

经典参考2Reference 2

The Authors – Cornell Book



Jon Kleinberg
Cornell
(Ph.D. from MIT)



Éva Tardos
Cornell University
(Hungarian mathematician, **Gödel Prize**)

- 1st Edition: 2006 – Coauthored with Éva Tardos
- 2nd Edition: 2013 – Sole Author

Jon Kleinberg's younger brother → Robert Kleinberg@Cornell

Chinese Version References

算法设计与分析(第三版). 王晓东.
电子工业出版社.2007年5月.



计算机算法导引: 设计与分析.
卢开澄. 清华大学出版社. 2006年.



中文版本参考

算法设计与分析(第三版). 王晓东.
电子工业出版社 .2007 年 5 月 .



计算机算法导引: 设计与分析. 卢
开澄. 清华大学出版社. 2006年.



Survey of Topics Introduced Worldwide

Totally 10 Topics

Algorithm Topics	Percentage
Algorithm Design and Analysis	95.2%
Graph Algorithms	92.9%
Approximation	88.1%
Dynamic Programming	83.3%
Greedy Approach	79.5%
Linear Programming	78.6%
Divide and Conquer	73.8%
Randomized Algorithm	73.8%
NP-Completeness	71.4%
Amortized Analysis	66.7%

全球性主题介绍调查

Totally 10 Topics

Algorithm Topics	Percentage
Algorithm Design and Analysis	95.2%
Graph Algorithms	92.9%
Approximation	88.1%
Dynamic Programming	83.3%
Greedy Approach	79.5%
Linear Programming	78.6%
Divide and Conquer	73.8%
Randomized Algorithm	73.8%
NP-Completeness	71.4%
Amortized Analysis	66.7%

Topics Introduced in Our Class

☐	序号 ▾	上课时间 ▾	上课内容 ▾
☐	1	第1周 星期4 第3节-第4节	基本概念和算法导引
☐	2	第2周 星期4 第3节-第4节	算法效率分析基础
☐	3	第3周 星期4 第3节-第4节	算法效率分析基础，蛮力法
☐	4	第4周 星期4 第3节-第4节	分治法思想、框架及案例
☐	5	第5周 星期4 第3节-第4节	减治、变治法思想及框架
☐	6	第6周 星期4 第3节-第4节	变治法案例、搜索算法简介
☐	7	第7周 星期4 第3节-第4节	回溯搜索算法及案例
☐	8	第8周 星期4 第3节-第4节	分支限界搜索算法及案例
☐	9	第9周 星期4 第3节-第4节	动态规划简介及案例1
☐	10	第10周 星期4 第3节-第4节	五一放假
☐	11	第11周 星期4 第3节-第4节	动态规划案例2
☐	12	第12周 星期4 第3节-第4节	动态规划案例3
☐	13	第13周 星期4 第3节-第4节	动态规划案例4及总结、拓展
☐	14	第14周 星期4 第3节-第4节	贪心算法简介及案例1
☐	15	第15周 星期4 第3节-第4节	贪心算法案例2
☐	16	第16周 星期4 第3节-第4节	算法复杂度分析P\NP问题
☐	17	第17周 星期4 第3节-第4节	习题课及总复习

我们课程中介绍的主题

☐	序号 ▾	上课时间 ▾	上课内容 ▾
☐	1	第1周 星期4 第3节-第4节	基本概念和算法导引
☐	2	第2周 星期4 第3节-第4节	算法效率分析基础
☐	3	第3周 星期4 第3节-第4节	算法效率分析基础，蛮力法
☐	4	第4周 星期4 第3节-第4节	分治法思想、框架及案例
☐	5	第5周 星期4 第3节-第4节	减治、变治法思想及框架
☐	6	第6周 星期4 第3节-第4节	变治法案例、搜索算法简介
☐	7	第7周 星期4 第3节-第4节	回溯搜索算法及案例
☐	8	第8周 星期4 第3节-第4节	分支限界搜索算法及案例
☐	9	第9周 星期4 第3节-第4节	动态规划简介及案例1
☐	10	第10周 星期4 第3节-第4节	五一放假
☐	11	第11周 星期4 第3节-第4节	动态规划案例2
☐	12	第12周 星期4 第3节-第4节	动态规划案例3
☐	13	第13周 星期4 第3节-第4节	动态规划案例4及总结、拓展
☐	14	第14周 星期4 第3节-第4节	贪心算法简介及案例1
☐	15	第15周 星期4 第3节-第4节	贪心算法案例2
☐	16	第16周 星期4 第3节-第4节	算法复杂度分析P\NP问题
☐	17	第17周 星期4 第3节-第4节	习题课及总复习

Examination

- Homework: 40%, 4 times(10%+10%+10%+10%)
- Final examination: 50%
- Attendance: 5% (being absent ≥ 5 times, you will fail this course)
- Class activity: 5% being active in class and answering questions correctly

考试

- 作业：40%，4次（10%+10%+10%+10%）
- 期末考试：50%
- 出勤：5%（缺勤 ≥ 5 次，你将这门课不及格）
- 课堂活动：5%（积极参与课堂并正确回答问题）

Course Prerequisite

Data Structure

C, Java or other programming languages

Discrete Mathematics

Advanced Mathematics

课程先修条件

数据结构

C语言、Java或其他编程语言

离散数学

高等数学

Why we learn Algorithm ?

- **Donald E. Knuth** Stanford Univ. Turing Award 1974
- *The Art of Computer Programming*

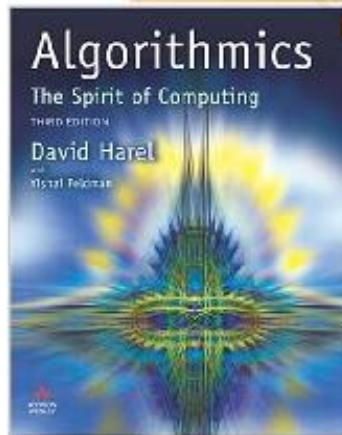
*Computer Science is the study of algorithms.
Cornerstone of computer science.
Programs will not exist without algorithms.*

- **Algorithmics: the Spirit of Computing**

Prof. David Harel Dean of Faculty of Mathematics and
Computer Science, the Weizmann Institute of Science

*Algorithmics is more than a branch of computer science. It is
the core of computer science, and, in all fairness, can be said
to be relevant to most of science, business, and technology.*

*Only when you teach your computer technologies,
you can get REAL control of it*



我们为什么要学习算法?

- Donald E. Knuth 斯坦福大学 转折奖 1974
- 《计算机程序设计艺术》

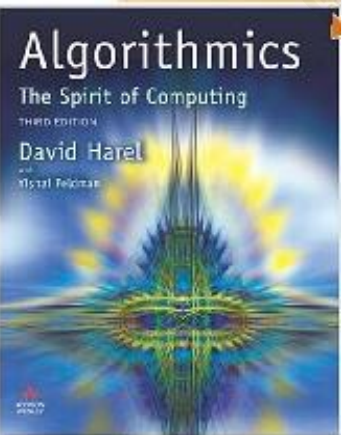
计算机科学是算法的研究。计算机科学的基础。
没有算法，程序将不存在。

- 算法学：计算的精髓

David Harel 教授 数学学院院长

计算机科学，魏茨曼科学研究所的算法学不仅仅是计算机科学的一个分支。
它是

计算机科学的核心，而且，说句公道话，它与大部分科学、商业和技术都有关联。只有当你教授计算机技术时，你才能真正掌控它



Why we learn Algorithm ?

❏ *Closely related to our lives*



❏ *Help to guide how to analyze and solve problems*

❏ *Help to develop the ability of analyzing and solving problems via computers*

我们为什么要学习算法?

❏ 与我们的生活密切相关



❏ 帮助指导如何分析和解决问题

❏ 帮助开发通过计算机分析和解决问题的能力

Applications of algorithms

■ Human Genome Project

- ✦ *identifying all the 100,000 genes in human DNA*
- ✦ *determining the sequences of that make up human DNA, the 3 billion chemical base pairs*
- ✦ *storing this information in databases*
- ✦ *developing tools for data analysis*
- ✦ *ideas and techniques in this course are used in the solution of these biological problems*
- ✦ *accomplish tasks while using resources efficiently*
- ✦ *Savings in time, human, machine, and money*



算法的应用 applications of algorithms

■ 人类基因组计划

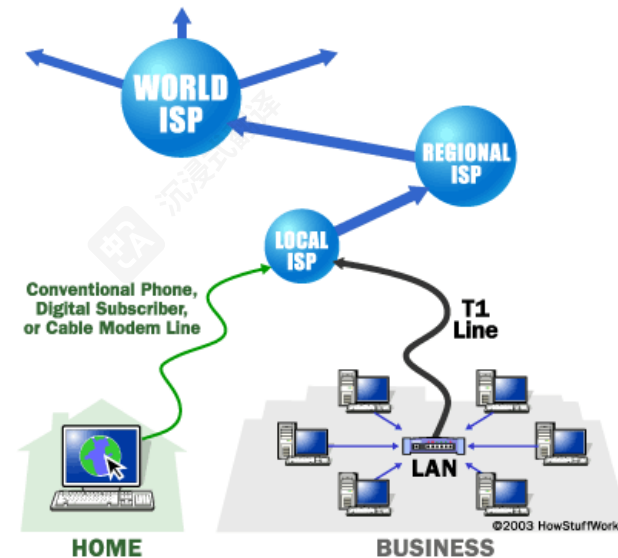
- ✦ 识别人类DNA中的所有10万个基因
- ✦ 确定构成人类DNA的序列，即30亿个化学碱基对
- ✦ 将此信息存储在数据库中
- ✦ 开发数据分析工具
- ✦ 本课程中的思想和技巧被用于解决这些生物学问题
- ✦ 在高效利用资源的同时完成任务
- ✦ 节省时间、人力、机器和金钱



Applications of algorithms

■ The Internet

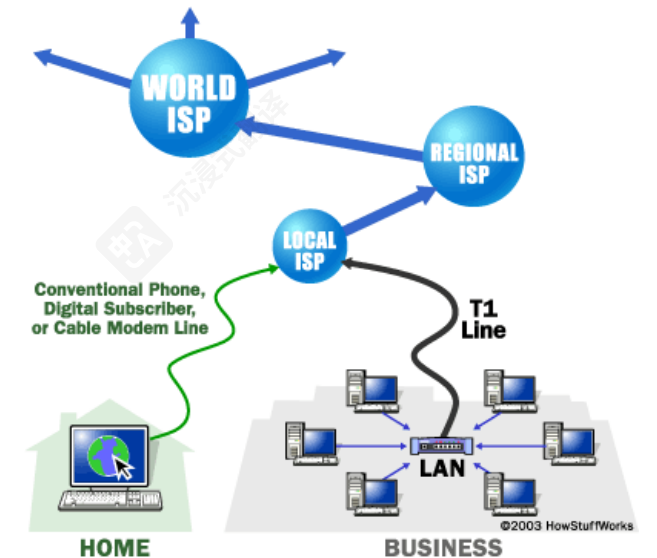
- ✦ quickly access and retrieve large amounts of information
- ✦ algorithms are employed to manage and manipulate this large volume of data
- ✦ e.g. finding good routes on which the data will travel
- ✦ e.g. information search engine



算法的应用

■ 互联网

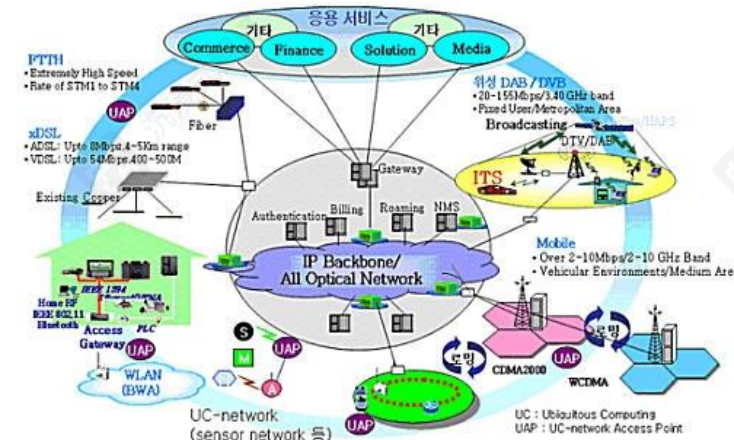
- ✦ 快速访问和检索大量信息
- ✦ 算法被用于管理和处理这些大量数据
- ✦ 例如，寻找数据将传输的良好路线，例如信息搜
- ✦ 索引



Applications of algorithms

■ Communications

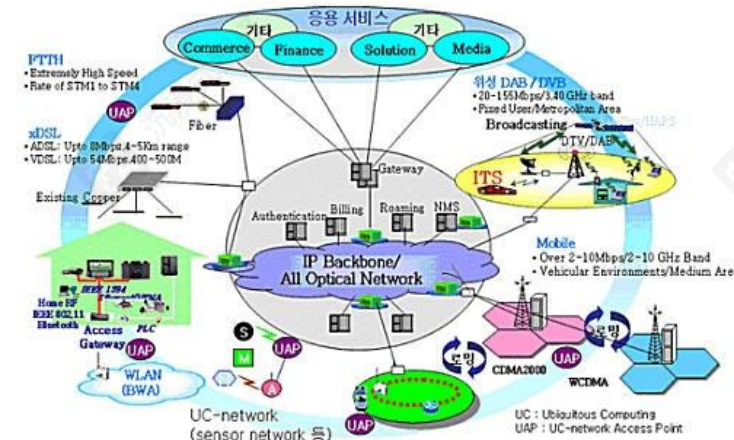
- *How to transmit multimedia data*
- *How to organize different information streams on the network*
- *How to storage data on the network*
- *multimedia information retrieval*



算法的应用

■ 通信

- 如何传输多媒体数据
- 如何在网络上组织不同的信息流 如何在网络上存储数据
- 多媒体信息检索



Applications of algorithms

■ Cryptography in e-commerce

- ✦ *to keep information such as credit card numbers, passwords, and bank statements private*
- ✦ *Public-key cryptography and digital signatures*



算法的应用

■ 电子商务中的密码学

- ✦ 以保护信用卡号码、密码和银行对账单等信息的安全
- ✦ 公钥密码学和数字签名



Applications of algorithms

■ In manufacturing and other commercial settings,

- ✦ *An oil company may wish to know where to place its wells in order to maximize its expected profit.*
- ✦ *An airline may wish to assign crews to flights in the least expensive way possible, making sure that each flight is covered*
- ✦ *An Internet service provider may wish to determine where to place additional resources in order to serve its customers more effectively.*

— *linear programming*

算法的应用 *Applications of algorithms*

■ 在制造业和其他商业环境中,

- ✦ 一家石油公司可能希望知道如何布置其油井以最大化预期利润。
- ✦ 航空公司可能希望以最低成本的方式为航班分配机组人员, 确保每趟航班都有人覆盖
- ✦ 互联网服务提供商可能希望确定在哪里放置额外资源, 以便更有效地为其客户提供服务。

— 线性规划

What we learn in this course ?

- ❏ Systematical **study of classical algorithms** in the computer science area
- ❏ **Master the typical techniques and methods** of algorithms design
- ❏ Abilities of **analyzing complexity of algorithms**
- ❏ Be able to **design algorithms** for simple or complex practical problems
- ❏ Try to make the algorithms efficient and effective to **enhance the quality of programming**.

我们在本课程中学到什么？

- ❏ 计算机科学领域的经典算法系统学习
- ❏ 掌握算法设计中的典型技术和方法
- ❏ 分析算法复杂度的能力
- ❏ 能够为简单或复杂的实际问题设计算法
- ❏ 努力使算法高效且有效，以提升编程质量。

Chapter 1: Introduction

- What's Algorithm
- Example of Algorithm
- Algorithm vs. Program
- Algorithmic problem solving
- Contents of Algorithm

第一章: 引言

- 什么是算法
- 算法示例
- 算法与程序
- 算法问题解决
- 算法内容

What's Algorithm

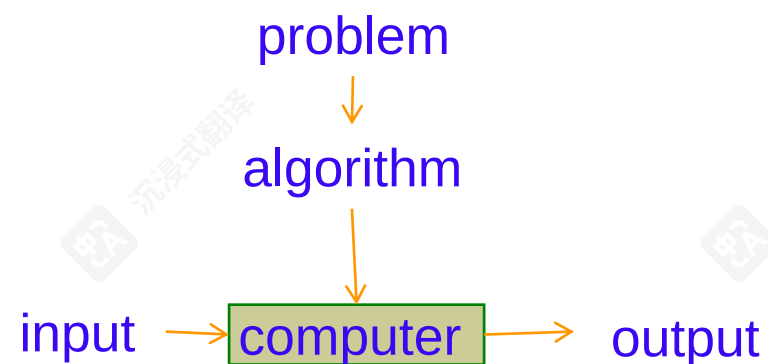
■ Notation

✦ The algorithm describes a specific **computational procedure** for solving a well-specified **computational problem**.

✦ The statement of the problem specifies in general terms the desired input/output relationship. An algorithm is for achieving that **input/output relationship**.

✦ Can achieve the desired **output** for any **specific legitimate** input in a **finite amount of time**.

✦ An algorithm is a **finite** sequence of **unambiguous instructions**



什么是算法

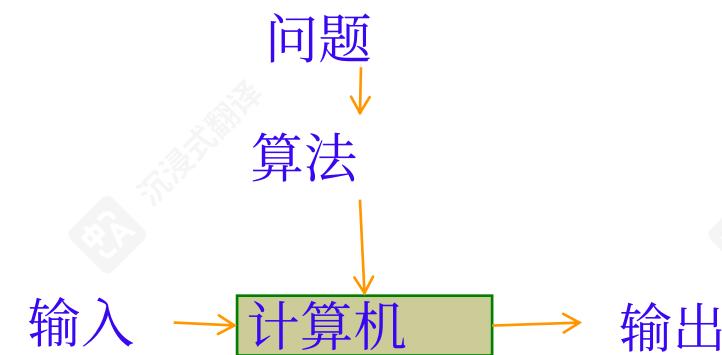
■ 符号

✦ 该算法描述了一种特定的计算过程，用于解决一个明确规定的计算问题。

✦ 问题的陈述以一般术语指定了期望的输入/输出关系。算法是为了实现这种输入/输出关系。

✦ 它能在有限时间内对任何特定的合法输入产生期望的输出。

✦ 算法是一个有限的不含糊的指令序列。



What's Algorithm

■ Properties of algorithms

✦ **Input:** 0 or more valid input values, to provide the initialization conditions

✦ **Output:**

- produce the correct output given a valid input
- **at least one value** is produced by the algorithm
- desired input/output relationship specified by the problem

✦ **Tangibility(确定性):**

- each instruction / each step is clearly
- precisely and unambiguously specified

Example: 不符合确定性的运算

- 5/0
- 将6或7与x相加
- 未赋值变量参与运算

算法是什么

■ 算法的性质

✦ 输入：0个或多个有效输入值，以提供初始化条件

✦ 输出：

- 给定有效输入时，产生正确的输出
- 算法至少产生一个值
- 问题指定的期望输入/输出关系

✦ 确定性 (Tangibility) :

- 每条指令 / 每个步骤都清晰
- 精确且明确地指定

示例：不符合确定性的运算

- 5/0
- 将6或7与x相加
- 未赋值变量参与运算

What's Algorithm

■ Properties of algorithms

✦ *Finiteness:*

- finite instructions,
- finite execution times for each instruction
- finite running time for each instruction

- ✦ *Feasibility:* could be precisely executed and effectively computable;
Steps must be sufficiently simple and basic.

算法是什么

■ 算法的性质

✦ 有穷性:

- 指令有限,
- 每条指令的执行时间有限
- 每条指令的运行时间有限

- ✦ 可行性: 可以精确执行且有效可计算; 步骤必须足够简单和基础。

What's Algorithm

■ Some points for algorithms

- ✦ Each step of an algorithm must be **unambiguous**.
- ✦ **Different algorithms** for a certain problem
- ✦ **Different representations** to describe a certain algorithm
- ✦ **Different ideas and different execution speed** for different algorithms

算法是什么

■ 算法的一些要点

- ✦ 算法的每一步都必须明确无误。
- ✦ 针对同一个问题，可以有不同的算法
- ✦ 可以用不同的表示方法来描述同一个算法
- ✦ 不同算法具有不同的思想和执行速度

Example of Algorithm

■ Problem: Computing the Greatest Common Divisor of two integers

✦ $\text{gcd}(m, n)$: the largest integer that divides both m and n

■ Algorithm I

✦ *Euclid's algorithm:*

$\text{gcd}(m, n) = \text{gcd}(n, m \bmod n)$ iteratively while $n \neq 0$

$\text{gcd}(m, 0) = m$

✦ *Natural language*

Step1: If $n = 0$, return the value of m as the answer and stop;
otherwise, proceed to Step 2.

Step2: Divide m by n and assign the value of the remainder to r .

Step 3: Assign the value of n to m and the value of r to n . Go to Step 1.

算法示例

■ 问题：计算两个整数的最大公约数

✦ $\text{gcd}(m, n)$: 能同时整除 m 和 n 的最大整数

■ 算法一

✦ *Euclid's algorithm:*

$\text{gcd}(m, n) = \text{gcd}(n, m \bmod n)$ iteratively while $n \neq 0$

$\text{gcd}(m, 0) = m$

✦ *Natural language*

步骤1: 如果 $n = 0$, 则返回 m 的值作为答案并停止; 否则,
继续步骤2。

步骤2: 将 m 除以 n , 并将余数的值赋给 r 。步骤3: 将 n 的值赋给 m , 将
 r 的值赋给 n 。转到步骤1。

Example of Algorithm

✦ Pseudocode

- A mixture of a natural language and programming language-like structures
- Precise and succinct
- Pseudocode in this course
 - omits declarations of variables
 - use indentation to show the scope of such statements as for, if, and while.
 - Use $\leftarrow$ for assignment

Algorithm Euclid(m, n)

//Computes gcd(m, n) by Euclid's algorithm

//Input: Two nonnegative, not-both-zero integers m and n

//Output: Greatest common divisor of m and n

while $n \neq 0$ do

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

算法示例

✦ 伪代码

- 自然语言与类似编程语言的结构混合
- 精确且简洁
- 本课程中的伪代码
 - 省略变量声明
 - 使用缩进来显示 for、if 和 while 等语句的作用范围。
 - 用于赋值 $\leftarrow$

Algorithm Euclid(m, n)

//Computes gcd(m, n) by Euclid's algorithm

//Input: Two nonnegative, not-both-zero integers m and n

//Output: Greatest common divisor of m and n

while $n \neq 0$ do

$m \bmod \leftarrow$ n

$m \leftarrow n$

$n \leftarrow r$

return m

Example of Algorithm

■ Algorithm II

✦ Consecutive Integer Algorithm

Step1: Assign the value of $\min\{m, n\}$ to t .

Step2: Divide m by t . If the remainder of this division is 0, go to Step3; otherwise, go to Step 4.

Step3: Divide n by t . If the remainder of this division is 0, return the value of t as the answer and stop;

otherwise, proceed to Step4.

Step4: Decrease the value of t by 1. Go to Step2.

算法示例

■ 算法 II

✦ 连续整数算法

步骤1: 将 $\min\{m, n\}$ 的值赋给 t 。

步骤2: 将 m 除以 t 。如果除法的余数为 0, 则转到步骤3; 否则, 转到步骤 4。

步骤3: 将 n 除以 t 。如果除法的余数为 0, 则返回 t 的值作为答案并停止;

否则, 继续执行步骤4。

步骤4: 将 t 的值减1。转到步骤2。

Example of Algorithm

■ Algorithm II

✦ Consecutive Integer Algorithm

```
Algorithm ConsecutiveInteger( $m, n$ )  
//使用连续整数检测法计算gcd( $m, n$ )  
//输入: 两个不全为0的非负整数 $m, n$   
//输出:  $m, n$ 的最大公约数  
if  $n=0$  return  $n$   
     $t=\min\{m, n\}$   
    while  $t>0$  do  
        if  $(m \bmod t) == 0$   
            if  $(n \bmod t) == 0$   
                return  $t$   
            else  $t=t-1$   
        else  $t=t-1$   
    return  $t$ 
```

算法示例

■ 算法 II

✦ 连续整数算法

```
Algorithm ConsecutiveInteger( $m, n$ )  
//使用连续整数检测法计算gcd( $m, n$ )  
//输入: 两个不全为0的非负整数 $m, n$   
//输出:  $m, n$ 的最大公约数  
if  $n=0$  return  $n$   
     $t=\min\{m, n\}$   
    while  $t>0$  do  
        if  $(m \bmod t) == 0$   
            if  $(n \bmod t) == 0$   
                return  $t$   
            else  $t=t-1$   
        else  $t=t-1$   
    返回  $t$ 
```


Example of Algorithm

■ Algorithm III ?

✦ Middle-school procedure

Step1: Find the prime factors of m. ?

Step2: Find the prime factors of n. ?

Step3: Identify all the common factors in the two prime expansions found in

Step1 and Step2. (If p is a common factor occurring P_m and P_n times in m and n , respectively, it should be repeated in $\min\{P_m, P_n\}$ times.)

Step4: Compute the product of all the common factors and return it as the gcd of the numbers given.

算法示例

■ 算法III?

✦ 中学流程

步骤1: 找出 m 的质因数。 ?

步骤2: 找出 n 的质因数。 ?

步骤3: 识别在步骤1和步骤2中找到的两个质因数展开式中的所有公因数。

步骤1和步骤2。 (如果 p 是一个公因数, 在 m 和 n 中分别出现 P_m 和 P_n 次, 则应在 $\min\{P_m, P_n\}$ 次中重复。) 步骤4: 计算所有公因数的乘积, 并将其作为给定数字的最大公约数返回。

Algorithm vs. Program

■ Similarity

- ✦ *Finite sequence of instructions*

■ Difference

✦ *Presentation:*

Algorithm —— *Nature language, pseudo code, flow charts*

Program —— *Coded using some specific programming language
Could be executed by some specific machine*

✦ *Execution:*

Algorithm —— *finite steps*

Program —— *could be infinitely executed*

e.g. Operating system

- *not an algorithm, but a program running in infinite circles*
- *each task could be viewed as subprogram according to specific algorithm*

算法与程序 vs. Program

■ 相似度

- ✦ 有限的指令序列

■ 差异

✦ 演示:

算法 —— 自然语言、伪代码、流程图程序 —— 使用特定编程语言编码可以在特定机器上执行

✦ 执行:

算法 —— 有限步骤

程序 —— 可以无限执行, 例如操作系统

- 不是算法, 而是一个在无限循环中运行的程序
- 根据特定算法, 每个任务都可以被视为子程序

Algorithm vs. Program

区别

定义:

Algorithm — a step by step outline or flowchart

Program — an implemented coding of a solution based on the algorithm

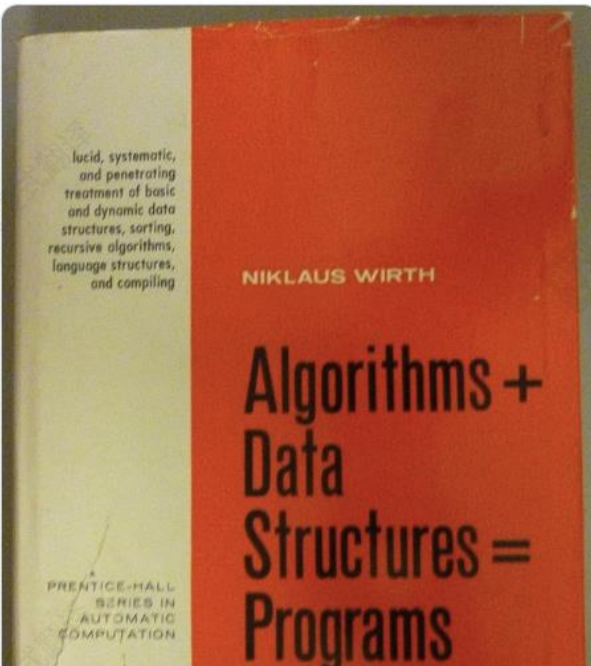
Algorithm + data structure = program

Jeff Dean (@JeffDean) · 2015
Very sad to hear. 😞 Niklaus Wirth made many enormous contributions to the field of computing. Pascal was the first language I used seriously (initially on the UCSD p-System and later via Turbo Pascal), and I got my hands on this great book that he wrote when I was in middle school. Later, I loved using Modula-2 for some things (and his indirect influence on Modula-3 was strong).

I also love the anecdote when asked about how to pronounce his name:

"Whereas Europeans generally pronounce my name the right way ('Niklaus Wirt'), Americans invariably mangle it into 'Nick-les Worth'. This is to say that Europeans call me by name, but Americans call me by value."

Niklaus Wirth"
翻译帖子



算法与程序

区别

定义:

算法 — 按部就班的轮廓或流程图，用于解决一个问题

程序 — 对问题解决方案的实现编码
基于算法

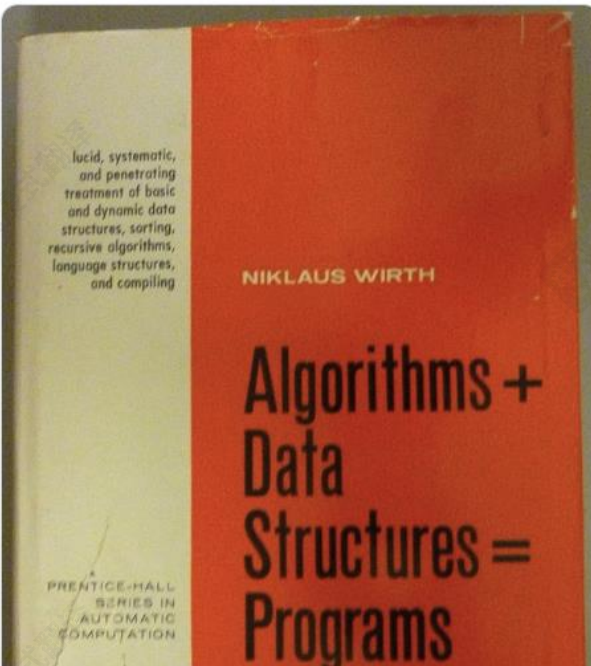
算法 + 数据结构 = 程序

Jeff Dean (@JeffDean) · 2015
Very sad to hear. 😞 Niklaus Wirth made many enormous contributions to the field of computing. Pascal was the first language I used seriously (initially on the UCSD p-System and later via Turbo Pascal), and I got my hands on this great book that he wrote when I was in middle school. Later, I loved using Modula-2 for some things (and his indirect influence on Modula-3 was strong).

I also love the anecdote when asked about how to pronounce his name:

"Whereas Europeans generally pronounce my name the right way ('Niklaus Wirt'), Americans invariably mangle it into 'Nick-les Worth'. This is to say that Europeans call me by name, but Americans call me by value."

Niklaus Wirth"
翻译帖子



Algorithmic problem solving

Algorithm Design and Analysis Process

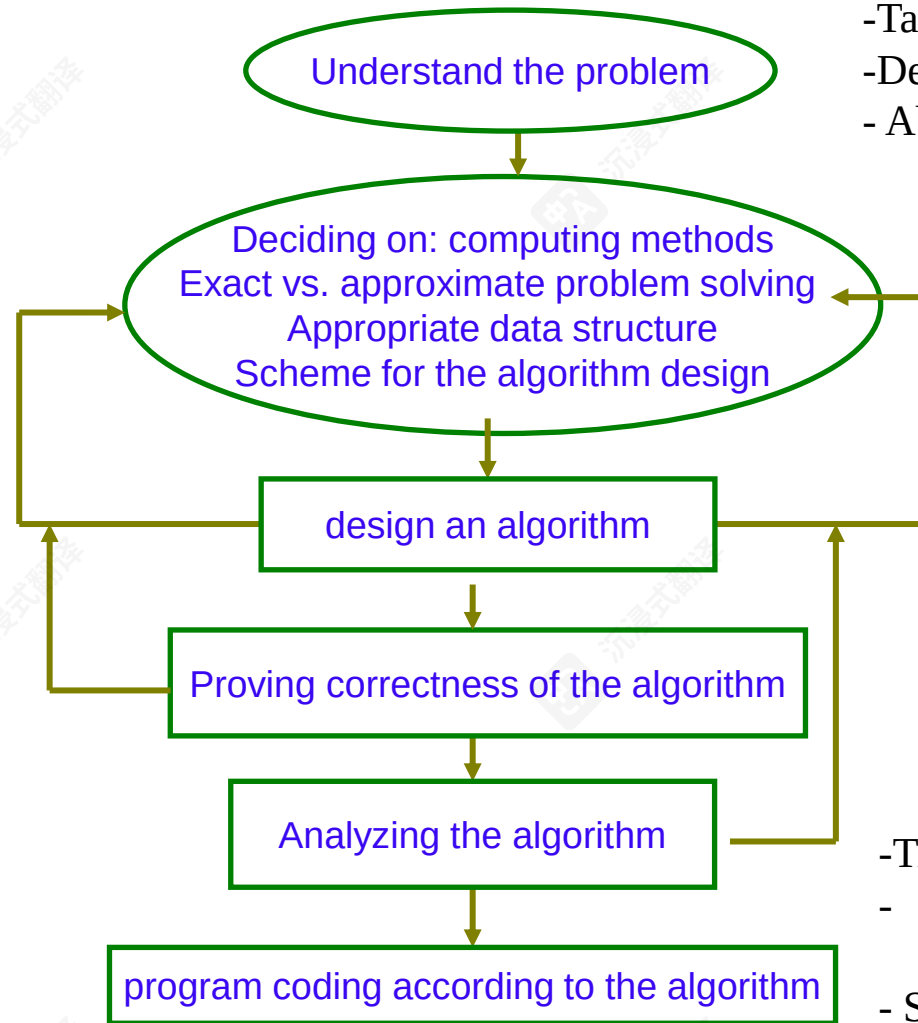
- Understand the problem description
- Try some examples manually
- Take into consideration special examples
- Define the input
- Abstract the problem and get its mathematical description

- Equipment performance
- Computing methods: sequential or parallel
- Exact solution is unavailable or speed is unacceptably low
- Algorithm + data structure = program

- Nature language
- pseudo code
- flow charts

- For every legal input, the algorithm will produce a desired output in finite time
- Mathematical Induction
- to prove its correctness or incorrectness ?

- Time efficiency : how fast the algorithm runs
- Space efficiency: how much extra memory the algorithm needs.
- Simplesness and commonness



算法问题解决

算法设计与分析过程

- 理解问题描述
- 手动尝试一些例子
- 考虑特殊情况
- 定义输入
- 抽象化问题并获取其数学描述

- 设备性能
- 计算方法：顺序或并行
- 无法获得精确解或速度不可接受

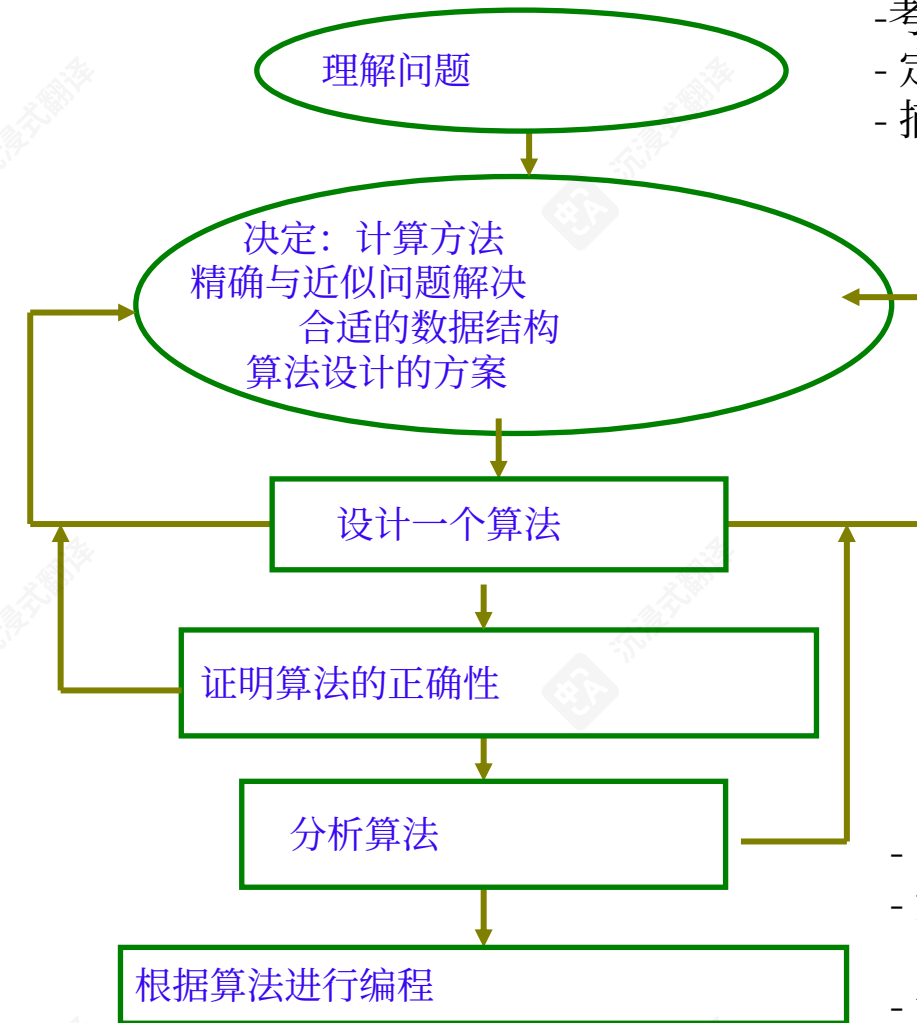
- 算法 + 数据结构 = 程序

- 自然语言
- 伪代码
- 流程图

- 对于每个合法输入，算法将在有限时间内产生期望输出
- 数学归纳法
- 要证明其正确性或错误性？

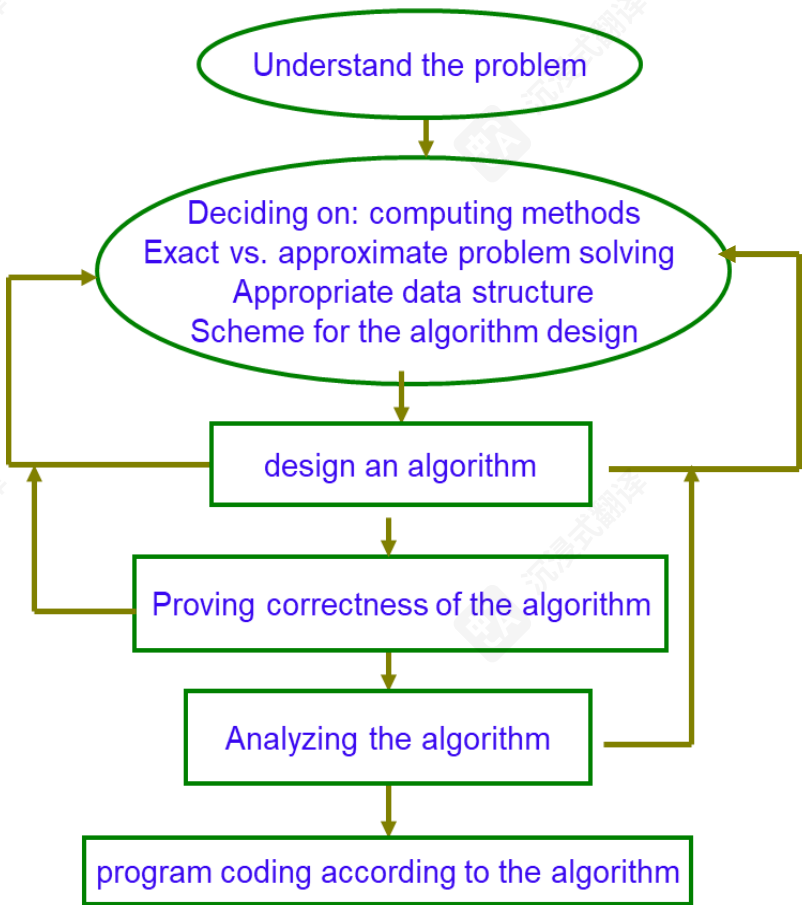
- 时间效率：算法运行有多快
- 空间效率：算法需要多少额外内存。

- 简洁性和普遍性



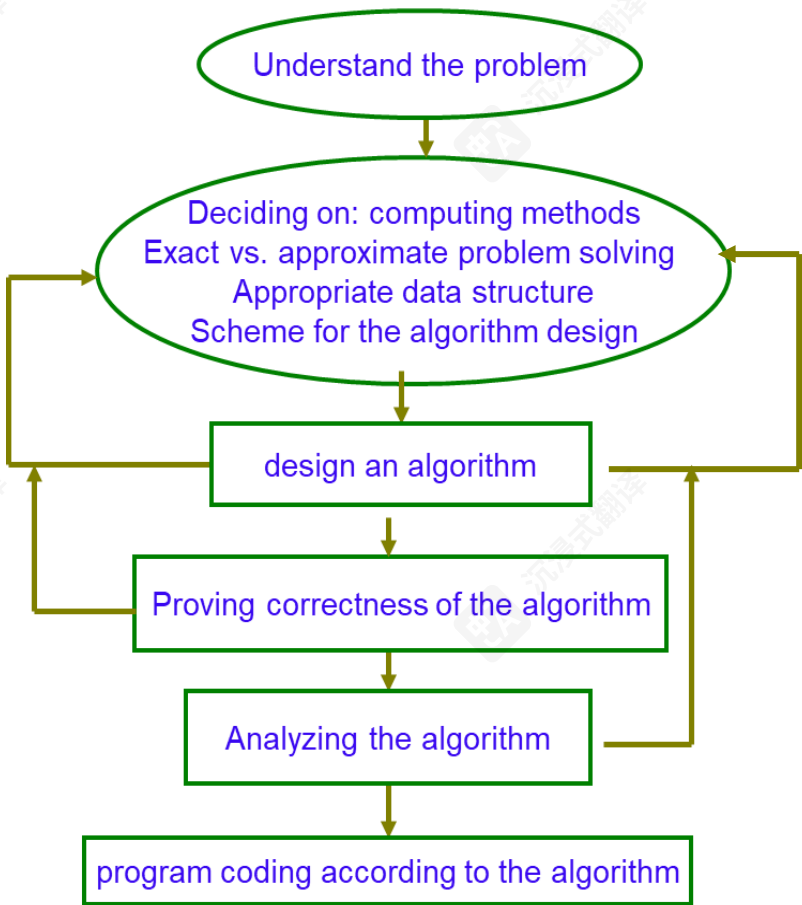
Algorithmic problem solving

Algorithm Design and Analysis Process



算法问题解决

Algorithm Design and Analysis Process



Contents of Algorithm

■ Algorithm Design Techniques/Strategies

- ✦ Brute force 蛮力法
- ✦ Divide and conquer 分治法
- ✦ Decrease and conquer 减治法
- ✦ Transform and conquer 变治法
- ✦ Greedy approach 贪心算法
- ✦ Dynamic programming 动态规划
- ✦ Back tracking 回溯法
- ✦ Branch and bound 分支界限法

算法内容

■ 算法设计技巧/策略

- ✦ 暴力法 蛮力法
- ✦ 分治法
- ✦ 减治法
- ✦ 变换与征服 变治法
- ✦ 贪婪算法 贪心算法
- ✦ 动态规划 动态规划
- ✦ 回溯法 回溯法
- ✦ 分支与界限 分支界限法

Contents of Algorithm

■ How to analyze algorithm efficiency

✦ How good is the algorithm?

- time efficiency
- space efficiency

✦ Does there exist a better algorithm?

- lower bounds
- optimality

算法内容 of Algorithm

■ 如何分析算法效率

✦ 算法有多好?

- 时间效率
- 空间效率

✦ 是否存在更好的算法?

- 下界
- 最优性

Contents of Algorithm

■ Important problem types

- ★ *sorting* 排序
- ★ *searching* 查找
- ★ *string processing* 串处理
- ★ *graph problems* 图问题
- ★ *combinatorial problems* 组合问题
- ★ *geometric problems* 几何问题
- ★ *numerical problems* 数值问题

算法内容

■ 重要问题类型

- ★ 排序 排序
- ★ 搜索 查找
- ★ 字符串处理 串处理
- ★ 图问题 图问题
- ★ 组合问题 组合问题
- ★ 几何问题 几何问题
- ★ 数值问题 数值问题

Contents of Algorithm

■ Fundamental data structures

✦ linear data structure

- array 数组
- linked list 单（双）链表
- string 串
- stack 栈
- queue 队列

✦ graph 图

✦ tree 树

✦ set and dictionary 集合

算法内容

■ 基本数据结构

✦ 线性数据结构

- 数组 数组
- 链表 单(双)链表
- 字符串 串
- 堆栈 栈
- 队列 队列

✦ 图 图

✦ tree 树

✦ 集合和字典 集合与字典

Summary

- ✦ 算法的定义：在有限时间内，对问题求解的一个清晰的指令序列。算法的每个输入确定了该算法求解问题的一个实例。
- ✦ 算法的特点：输入，输出，确定性，有穷性，和可行性。
- ✦ 算法可以用自然语言或者伪代码表示，或计算机程序实现
- ✦ 一个好的算法常常是不懈努力和反复修改的结果。
- ✦ 算法操作的是数据，所以数据结构很重要。

摘要

- ✦ 算法的定义:在有限时间内,对问题求解的一个清晰的指令序列。算法的每个输入确定了该算法求解问题的一个实例。
- ✦ 算法的特点：输入、输出、确定性、有穷性和可行性。
- ✦ 算法可以用自然语言或者伪代码表示，或计算机程序实现
- ✦ 一个优秀的算法往往是持续努力和不断改进的成果。
- ✦ 算法操作的是数据，所以数据结构很重要。

思考题

证明：令 $d = \gcd(m, n)$ ，则 $d|m$ 且 $d|n$ 。设 $m = kn + r$ ($0 \leq r < n$)，则 $d|(kn+r)$ 。又有 $d|n$ ，因此 $d|kn$ ，所以有 $d|r$ 。即我们由 $d|m$ 且 $d|n$ 这个前提可以得出 $d|r$ 。换就话说，我们也可以说成由 $d|m$ 且 $d|n$ 这个前提推出了 $d|r$ 。使用类似的推理过程同样可以得到：由 $d|n$ 且 $d|r$ 可以推出 $d|m$ 且 $d|n$ 。这里的 r 即 $m \bmod n$ 。因此我们可以说 $\gcd(m, n) = \gcd(n, m \bmod n)$ 是双向成立的，命题得证。

1. Prove the equality $\gcd(m, n) = \gcd(n, m \bmod n)$ for every pair of positive integers m and n .

<http://blog.csdn.net/deserthero2013/article/details/51161696>

1. What does Euclid's algorithm do for a pair of numbers in which the first number is smaller than the second one? What is the largest number of times this can happen during the algorithm's execution on such an input?

思考题

证明：令 $d = \gcd(m, n)$ ，则 $d|m$ 且 $d|n$ 。设 $m = kn + r$ ($0 \leq r < n$)，则 $d|(kn+r)$ 。又有 $d|n$ ，因此 $d|kn$ ，所以有 $d|r$ 。即我们由 $d|m$ 且 $d|n$ 这个前提可以得出 $d|r$ 。换就话说，我们也可以说成由 $d|m$ 且 $d|n$ 这个前提推出了 $d|r$ 。使用类似的推理过程同样可以得到：由 $d|n$ 且 $d|r$ 可以推出 $d|m$ 且 $d|n$ 。这里的 r 即 $m \bmod n$ 。因此我们可以说 $\gcd(m, n) = \gcd(n, m \bmod n)$ 是双向成立的，命题得证。

1. 证明对于任意正整数对 m 和 n ， $\gcd(m, n) = \gcd(n, m \bmod n)$ 的等式成立。
of positive integers m and n .

<http://blog.csdn.net/deserthero2013/article/details/51161696>

1. 欧几里得算法对于第一数小于第二数的一对数做什么？
在算法对这种输入执行过程中，这种情况最多可以发生多少次？

上机练习

■ 1-1. Computing $\gcd(m, n)$

- 1) Compose a program using Euclid's algorithm
- 2) Compose a program using Consecutive Integer Algorithm
- 3) Find $\gcd(31415, 14142)$ by applying Euclid's algorithm
- 4) Estimate how many time faster it will be to find $\gcd(31415, 14142)$ by Euclid's algorithm compared with the algorithm based on checking consecutive integers from $\min\{m, n\}$ down to $\gcd(m, n)$

■ 1-2. find the binary representation of a positive decimal integer

Compose a program

上机练习

■ 1-1. 计算 $\gcd(m, n)$

- 1) 使用欧几里得算法编写程序
- 2) 使用连续整数算法编写程序
- 3) 通过应用欧几里得算法求 $\gcd(31415, 14142)$
- 4) 估计使用欧几里得算法求 $\gcd(31415, 14142)$ 比基于从 $\min\{m, n\}$ 检查到 $\gcd(m, n)$ 的连续整数算法快多少时间

■ 1-2. 查找一个正十进制整数的二进制表示 编写一个程序

上机练习

■ 1-3. Element uniqueness problem

- 1) a) Compose a program using *UniqueElement* algorithm on P63
b) Check its efficiency in worst case, best case, and average case, in your program
- 2) a) Compose a program using the method in which the array is sorted firstly
b) Check its efficiency in worst case, best case, and average case, in your program

上机练习

■ 1-3. 元素唯一性问题

- 1) a) 使用 UniqueElement 算法（P63 上）编写程序
b) 在你的程序中检查其在最坏情况、最好情况和平均情况下的效率
- 2) a) 使用首先对数组进行排序的方法编写程序
b) 在你的程序中检查其在最坏情况、最好情况和平均情况下的效率

代码要求

■ 1-1. GCD (1)

```
int gcd_Euclid (int m, int n)
{ // computes the greatest common divisor of two integers m and n
  using Euclid algorithm;
  // Input: two integers;
  // Output: their GCD

}
```

假定 m, n 都是自然数, 使用欧几里德算法求 m, n 的最大公约数, 作为返回值;

注意: 特殊情况 $m < n$

代码要求

■ 1-1. 最大公约数 (1)

```
int gcd_Euclid (int m, int n)
{ // 计算 m 和 n 两个整数的最大公约数
  using Euclid algorithm;
  // Input: two integers;
  // Output: their GCD

}
```

假设 m, n 都是自然数, 使用欧几里德算法求 m, n 的最大公约数, 作为返回值;

注意: 特殊情况 $m < n$

代码要求

■ 1-1. GCD (2)

```
int gcd_ConsecutInteger (int m, int n)
{ // computes the greatest common divisor of two integers m and n
  using Consecutive Integer Algorithm ;
    // Input: two integers;
    // Output: their GCD

}
```

假定 m, n 都是自然数，使用连续整数递减法求 m, n 的最大公约数，作为返回值；

注意：特殊情况 $m < n$

代码要求

■ 1-1. 最大公约数 (2)

```
int gcd_ConsecutInteger (int m, int n)
{ // 计算两个整数 m 和 n 的最大公约数
  using Consecutive Integer Algorithm ;
    // Input: two integers;
    // Output: their GCD

}
```

假设 m, n 都是自然数,使用连续整数递减法求 m, n 的最大公约数，作为返回值；

注意:特殊情况 $m < n$

代码要求

■ 1-2. binary representation of a positive decimal integer

```
int convert_decimal_to_binary(int dec_number)
{ // find the binary representation of a positive decimal integer;
  //Input: a positive decimal integer;
  //Output: binary integer;

}
```

结果输出到屏幕上（cout, printf），结果输出到一行
注意：要求不可以有前置的0，例如，结果不能是 00001111000
而是 1111000

代码要求

■ 1-2. 正整数的小数二进制表示

```
int convert_decimal_to_binary(int dec_number)
{ // find the binary representation of a positive decimal integer;
  //Input: a positive decimal integer;
  //Output: binary integer;

}
```

结果输出到屏幕上(cout,printf),结果输出到一行 注意:要求不
可以有前置的0,例如,结果不能是 00001111000 而是 1111000

代码要求

■ 1-3. Element uniqueness problem (1)

```
bool UniqueElement (int A[0,..., n-1], int size)
{ // Determines whether all the elements in a given array A are
  distinct using the definition based algorithm;
  // Input: an array A[0,..., n-1]) ;
  // Output: returns true if all the elements are distinct, and false
  otherwise;

}
```

代码要求

■ 1-3. 元素唯一性问题 (1)

```
bool UniqueElement (int A[0,..., n-1], int size)
{ // 使用基于定义的算法判断给定数组A中的所有元素是
  否唯一;
  // Input: an array A[0,..., n-1]) ;
  // 输出: 如果所有元素都唯一, 则返回true, 否则返回false
  otherwise;

}
```

代码要求

■ 1-3. Element uniqueness problem (2)

```
bool UniqueElement_sort (int A[0,..., n-1], int size)
{ // Determines whether all the elements in a given array A are
  distinct using sorting firstly;
  // Input: an array A[0,..., n-1]) ;
  // Output: returns true if all the elements are distinct, and false
  otherwise;

}
```

代码要求

■ 1-3. 元素唯一性问题 (2)

```
bool UniqueElement_sort (int A[0,..., n-1], int size)
{ // 首先通过排序, 判断给定数组A中的所有元素是否互
  不相同;
  // Input: an array A[0,..., n-1]) ;
  // 输出: 如果所有元素互不相同, 则返回true, 否则返回false
  otherwise;

}
```

代码要求

提交代码注意:

所有算法写入到一个c/cpp文件中,

文件命名 10xxxx_姓名_algo_assignment1.c

代码要求

提交代码注意:

所有算法写入到一个c/cpp文件中,

文件命名 10xxxx_姓名_algo_assignment1.c

文档要求

对三个练习分别给出

1. 算法本身的思路，可以用伪代码或者自然语言描述
2. 程序中用于计算算法复杂度的方法，用文字或者公式描述清楚

文档要求

对三个练习分别给出

1. 算法本身的思路，可以用伪代码或者自然语言描述
2. 程序中用于计算算法复杂度的方法,用文字或者公式描述清楚